Computer Vision Project Report (Plan)

License Plate Detection

Mikhail Krylov, Andrei Polevoi, Maximilian Mifsud Bonici, Evgenii Khovalyg

October 6, 2026

1 Project scope

Objective. Build and evaluate a two-stage pipeline that (a) *localizes* a license plate in a traffic image and (b) *reads* the characters on it.

Exact task.

- **Stage 1 – Detection:** single-class object detection of the license plate region, evaluated under varying viewing (tilt) angles and plate sizes/distances.
- **Stage 2 – Recognition (OCR):** read the plate string from the cropped region. Character set includes Chinese province characters, Latin letters, Arabic digits. Recognition is evaluated under two protocols: **OCR-only** (ground-truth plate crops, so detection errors do not leak into recognition) and **end-to-end** (crops from Stage 1 detection boxes).
- **Confidence policy:** each predicted plate is assigned a confidence score (mean per-character probability; product / sum of log-probabilities over characters as alternatives). A calibrated threshold on this score decides "read" vs. "abstain"; we report the accuracy–coverage curve (accuracy among non-abstained images vs. coverage, i.e., the fraction of images we do not abstain from), not just top-1 accuracy.

Models / settings planned.

- Detection: `https://huggingface.co/morsetechlab/yolov11-license-plate-detection`, size `s` used both as a pretrained baseline and after fine-tuning.
- Recognition: **CRNN** (CNN backbone + BiLSTM + CTC) for the read step; a standard pre-trained OCR engine used as the second baseline.
- Preprocessing ablation: none / grayscale+contrast / Canny edge, applied before OCR.

Research / engineering question. *How much do viewing angle, plate pixel size, and a simple preprocessing step change detection IoU and OCR accuracy (in both OCR-only and end-to-end protocols), and at what confidence threshold does the pipeline become reliable enough to abstain instead of fail?*

2 Dataset preparation

- **Dataset: CCPD** (Chinese license-plate dataset) — provides images with view angle, plate bounding box, plate size, and character-level annotations.

- **Subset:** a **5,000-image** subset sampled to **preserve the original distribution** of plate size and tilt angle so results reflect real conditions rather than an easy subset.
- **Preprocessing / derived data:**
  - Original-resolution images (primary).
  - **Downscaled variants** of each image, grouped into **size buckets** (e.g., large / medium / small by plate pixel height) to study the big-vs-small-plate effect.
- **Split: 3,503 train / 704 val / 777 test** ($\approx$ 70/14/16). Splits are made at the *image* level so the same image and its downscaled variants never appear in more than one split to avoid data leakage.

3 Experimental plan

#	Factor changed	Baseline → Condition	Metric
E1	Training data	Pretrained YOLOv11 → YOLOv11 fine-tuned on our train split	mAP@0.5, F1@IoU=0.5
E2	OCR model (OCR-only)	Pretrained OCR engine → CRNN fine-tuned on CCPD chars, on **ground-truth crops**	char accuracy, plate exact-match, P/R/F1
E2b	OCR model (end-to-end)	same models, on **detected crops** from Stage 1	char accuracy, plate exact-match, P/R/F1; Δ vs. OCR-only
E3	Preprocessing (ablation)	none / gray+contrast / Canny, before OCR	OCR accuracy, Δ per step
E4	Plate size (sensitivity)	large / medium / small buckets	IoU & OCR acc. per bucket
E5	Tilt angle (sensitivity)	binned by view angle	IoU & OCR acc. per bin
E6	Confidence threshold	per-plate score: mean char prob. (alt: $\sum_c \log p_c$); sweep threshold τ	accuracy@τ, coverage (non-abstain rate), accuracy–coverage curve

Table 1: Planned experiments. One major factor is changed at a time; OCR (E2/E2b) is evaluated both on ground-truth crops (OCR-only) and end-to-end; E3 is the ablation, E4/E5 are sensitivity analyses.

4 Implementation plan

Stack: Python 3.10, **PyTorch**, **OpenCV**, `ultralytics` (YOLOv11), a pretrained OCR engine EasyOCR, Pandas, Matplotlib.

Steps.

1. **Environment:** create a pinned `requirements.txt` / conda env.
2. **Data pipeline:** download CCPD → build the 5,000 stratified subset → generate downscaled size-bucket variants → write disjoint train/val/test splits.

3. **Detection:** evaluate baseline `morsetechlab/yolov11-license-plate-detection` on val/test; fine-tune on train, re-evaluate.

4. **Recognition:** crop plates using ground-truth boxes from the dataset (OCR-only protocol) → build the character image set → train the CRNN (CTC loss) on train, select epoch on val, evaluate on test.

5. **Preprocessing module (E3):** none / gray+contrast / Canny front-end for the OCR stage.

6. **Pipeline integration:** detection → crop → preprocess → OCR → confidence gating; end-to-end evaluation with detected crops (E2b); compute per-plate confidence (mean char probability / $\sum_c \log p_c$), sweep τ, report accuracy–coverage curve (E6); log per-stage latency and peak memory.

7. **Evaluation & reporting:** compute all metrics, produce the one comparison table/figure, dump the ≥ 3 failure cases, and verify reproducibility by re-running from a clean clone.

5 Team responsibilities *(proposed — confirm with group)*

- **Andrei Polevoi — Data & preprocessing:** CCPD subset, size buckets, splits/leakage check, label export, preprocessing module.
  a.polevoi@innopolis.university
- **Maximilian Mifsud Bonici — Detection:** YOLOv11 baseline + fine-tuning, IoU/mAP evaluation, detection failure cases.
  m.mifsudbonici@innopolis.university
- **Evgenii Khovalyg — Recognition:** CRNN training + OCR baseline, character/plate metrics, Chinese-character fine-tuning.
  e.khovalyg@innopolis.university
- **Mikhail Krylov — Pipeline & evaluation:** end-to-end integration, confidence thresholds, latency/memory, sensitivity tables, failure-analysis write-up, report & demo.
  m.krylov@innopolis.university